Table of Contents

1. Introduction 3

2. Problem Statement 3

3. Objectives 4

4. Detailed Features 4

4.1 Student Dashboard: 4

4.2 Educator Dashboard: 5

4.3 Assignment Management: 5

4.4 User Authentication and Role-Based Access: 5

4.5 Communication Tools and Analytics: 5

5. Personas and Scenarios 5

6. Technology Stack Justification 6

7. System Architecture and Workflow 7

8. Version Control and Collaboration 7

9. Work Allocation among team members 7

10. Project Planning and Gantt Chart 9

11 Conclusion 9

12 Appendix 10

1. Introduction

In today's academic environment, students and educators face numerous challenges in managing coursework, assignments, and schedules efficiently. With the increasing number of courses, coupled with tight deadlines and extensive learning materials, traditional methods of managing academic tasks, such as notebooks, spreadsheets, or informal reminders, are often insufficient. These manual approaches can lead to missed deadlines, disorganization, and increased stress for students, while educators may find themselves spending excessive time on administrative tasks rather than focusing on teaching and mentoring.

The Study Management System (SMS) is designed to address these challenges by providing a centralized, web-based platform that enables effective management of academic activities. The primary goal of the system is to assist students in organizing their study schedules, tracking assignments, and monitoring their academic progress while simultaneously offering educators tools to create, manage, and evaluate student work. By providing a unified interface for students and educators, SMS aims to streamline communication, improve organization, and enhance overall academic performance.

The system leverages modern web development technologies, ensuring it is scalable, secure, and user-friendly. The front-end will be developed using Next.js, a framework that allows for server-side rendering, dynamic content, and reusable React components. The back-end will use Node.js with Express.js, enabling efficient handling of HTTP requests, API routing, and business logic processing. Data will be stored in a MySQL relational database to ensure structured, secure, and consistent storage. Version control will be maintained using Git, supporting collaborative development, while deployment will be containerized using Docker to ensure portability and consistency across environments.

2. Problem Statement

Students often face difficulties in managing multiple courses, deadlines, and assignments simultaneously. They may struggle with prioritizing tasks, remembering deadlines, and maintaining a structured study schedule. These challenges can result in incomplete assignments, late submissions, and suboptimal academic performance. The lack of centralized tracking tools makes it difficult for students to monitor their progress effectively or plan their study routines strategically. Additionally, the absence of timely reminders or automated notifications exacerbates the likelihood of missed deadlines, adding unnecessary stress.

Educators also encounter significant challenges in managing student performance and monitoring assignments. In large classes, tracking individual submissions, ensuring timely grading, and providing constructive feedback can be time-consuming. Manual methods of managing assignments, such as

spreadsheets or emails, are prone to errors, delay, and miscommunication. Educators may also struggle to identify students who are falling behind, making it difficult to provide targeted support or interventions.

The cumulative effect of these issues is a lack of organization, reduced productivity, and increased stress for both students and educators. A centralized digital system is essential to automate tracking, provide timely notifications, and generate insights into academic performance. By offering a structured and organized approach to managing assignments and study schedules, the Study Management System aims to mitigate these challenges and support an efficient, effective academic workflow.

3. Objectives

- **Enhance Student Organization:** Provide a centralized dashboard for students to track assignments, deadlines, and study schedules, with task tracking and reminder functionalities to improve time management and academic performance.
- **Simplify Educator Management:** Enable educators to create, edit, and monitor assignments, track submissions in real-time, provide feedback, and access analytics to support data-driven decision-making.
- **Implement Role-Based Access Control:** Ensure that students and educators have access only to functionalities relevant to their roles, maintaining data security, privacy, and authorized access.
- **Provide Data Analytics and Reporting:** Generate insights on student performance, assignment completion rates, and engagement metrics to help educators identify trends and intervene when necessary.
- **Ensure Scalable Deployment and Maintenance:** Use Docker containerization to maintain consistent environments across development, testing, and production, allowing easy updates and future scalability.

4. Detailed Features

4.1 Student Dashboard:

- Displays pending assignments, upcoming deadlines, and study schedules.
- Allows marking tasks as complete and visualizing them in a calendar view.
- Sends notifications and reminders for upcoming deadlines to improve planning and productivity.

4.2 Educator Dashboard:

- Enables creating, updating, and monitoring assignments.
- Tracks student submissions, provides feedback, and visualizes performance through analytics.
- Supports uploading study materials and resources in a centralized location.

4.3 Assignment Management:

- Upload assignments and set deadlines.
- Automatic reminder notifications for students.
- Version tracking to ensure the latest assignment updates are available.

4.4 User Authentication and Role-Based Access:

- Secure login with encrypted passwords and session management.
- Role-based permissions ensure access only to relevant features for students and educators.

4.5 Communication Tools and Analytics:

- In-app messaging for queries, announcements, and clarifications.
- Reports and analytics for visualizing student performance, assignment trends, and course engagement.

5. Personas and Scenarios

Persona 1 – Student: Ananya Sharma

Name: Ananya Sharma

Age & Profile: 21-year-old undergraduate student

Goals: Organize her study schedule, track assignments, and improve academic performance

Challenges: Managing multiple assignments and deadlines, dealing with unorganized study materials, and avoiding missed deadlines

How the Study Management System (SMS) Helps:

Access all assignments and deadlines via a centralized dashboard

Receive notifications and reminders for upcoming tasks

Update progress for each assignment

Use calendar view to plan weekly study schedules efficiently

Benefits: Reduces stress, ensures timely completion of assignments, and improves academic performance

Persona 2 – Educator: Prof. Ravi Kumar

Name: Prof. Ravi Kumar

Age & Profile: 45-year-old lecturer teaching multiple courses

Goals: Efficiently manage courses, monitor student performance, and provide timely feedback

Challenges: Tracking submissions, providing feedback on time, and monitoring student engagement

How the Study Management System (SMS) Helps:

Upload assignments and study materials

Monitor student submissions in real-time

Provide feedback through the platform

Access analytics on student progress to identify students needing additional support

Benefits: Improves course management efficiency and enhances the overall learning experience for students

6. Technology Stack Justification

The Study Management System employs a modern web technology stack to ensure scalability, efficiency, and ease of maintenance. The front-end will be developed using Next.js, a React framework that allows server-side rendering, dynamic content generation, and reusable UI components. This choice ensures optimal performance, responsiveness, and an enhanced user experience.

The back-end will utilize Node.js with Express.js to handle HTTP requests, business logic, and API routing. Express.js simplifies server-side programming and facilitates communication with the front-end through RESTful APIs.

Data will be stored in a MySQL database, which offers relational storage, secure transactions, and efficient querying capabilities. The database will manage structured data such as users, courses, assignments, and submissions.

Version control will be managed using Git, allowing multiple team members to collaborate, track code changes, and manage feature development systematically. Docker will be employed for deployment, containerizing front-end, back-end, and database services to ensure reproducibility, consistency, and portability across different environments.

7. System Architecture and Workflow

The system architecture consists of three primary layers: the front-end, back-end, and database. The Next.js front-end manages user interactions, renders dashboards, and displays analytics. The Node.js back-end processes API requests from the front-end, handles authentication, and performs business logic. The MySQL database stores all application data securely.

The workflow begins when a user logs in. Students access their dashboard, view assignments, and update progress. These actions trigger back-end API calls that update the database and provide real-time feedback. Educators can create assignments, monitor submissions, and access performance analytics, all of which are processed by the back-end and stored in the database.

This architecture supports scalability, modularity, and future enhancements, including AI-based analytics or mobile app integration.

8. Version Control and Collaboration

Git will serve as the version control system for the project, enabling team members to work collaboratively. Feature branches will be used for adding new functionalities, while pull requests and code reviews will maintain code quality. Git also allows tracking of changes, rollback of updates if necessary, and seamless integration of contributions from all team members. Continuous integration can be implemented in future iterations to automate testing and deployment.

9. Work Allocation among team members

- **Kanukuntla Ranadheer (Team Leader – A00086685):**

  Overall project coordination and planning

Finalizing project requirements and objectives

Supervising front-end and back-end development

Ensuring timely progress and adherence to deadlines

Overseeing deployment and Docker setup

- **Kethineni Neeraj Krishna (A00076167):**

  Development of the front-end using Next.js

  Implementing user interfaces, dashboards, and calendar views

  Integrating front-end with back-end APIs

  Testing front-end functionalities and ensuring responsive design

- **Venkatasubbaiah Chowdary Kunku (A00075088):**

  Development of the back-end using Node.js and Express.js

  Creating API endpoints for assignments, users, and analytics

  Implementing user authentication and role-based access control

  Connecting the back-end with the MySQL database

- **Bhargava Sai Bellamkonda (A00073767):**

  Database design and management using MySQL

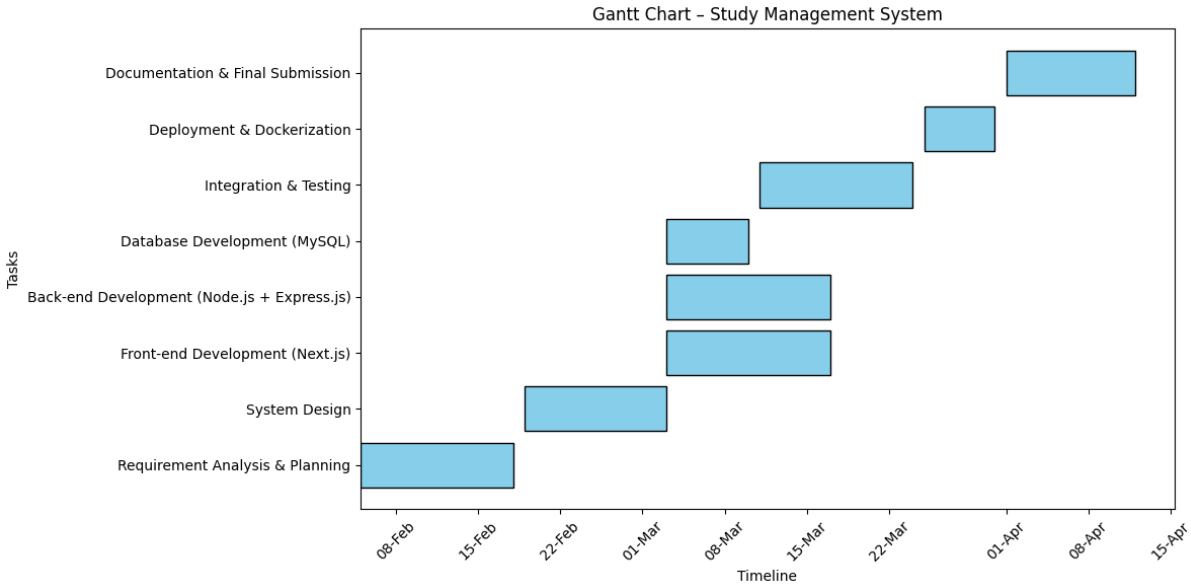
  Maintaining data integrity and creating relational schemas

  Supporting back-end integration and query optimization

  Assisting in reporting, analytics, and system testing

10. Project Planning and Gantt Chart

Effective project planning is essential to ensure that the Study Management System (SMS) is developed efficiently, meets deadlines, and delivers all functionalities as intended. The project will follow a phased approach, dividing tasks among team members and setting realistic timelines for each phase.



11 Conclusion

The Study Management System addresses key challenges faced by students and educators by providing a centralized, web-based platform for assignment management, study planning, and performance analytics. Leveraging modern technologies like Next.js, Node.js, Express.js, MySQL, Git, and Docker ensures scalability, maintainability, and user-friendly operation. The system is poised to improve academic efficiency, reduce administrative burdens, and support data-driven educational insights.

12 Appendix

Code of Conduct

1. All team members must communicate respectfully, value diverse perspectives, and maintain a professional attitude in all interactions.
2. Each member is responsible for completing assigned tasks on time and to the highest quality standards.
3. Team members are accountable for their work, including coding, testing, database management, and documentation.
4. Active participation in meetings, discussions, and decision-making is required, and members must assist each other whenever needed.
5. All contributions must be original. Plagiarism, falsification, or unethical practices are strictly prohibited.
6. Project data, code, database credentials, and design documents must remain confidential and not shared externally without consent.
7. Team members should provide clear, timely, and constructive communication to ensure smooth progress and coordination.
8. Members must respect deadlines, attend scheduled meetings, and provide regular updates on task progress.
9. The team will follow coding, documentation, and version control (Git) standards consistently throughout the project.
10. All work, including code, database schemas, and front-end designs, must undergo testing and review to ensure high quality.
11. Members are encouraged to learn new tools and technologies relevant to the project and apply feedback for improvement.
12. Any disagreements will be addressed calmly, constructively, and collaboratively to maintain team harmony and project progress.

Agreement of Participation

All team members agree to actively participate in the development and completion of the Study Management System project, taking responsibility for their assigned tasks and ensuring timely delivery. Each member commits to collaborating respectfully, maintaining professional conduct, and supporting the team to achieve project objectives. Confidentiality of all project data, code, and design documents will be upheld, and all contributions will be properly documented using Git. Conflicts or disagreements will be resolved constructively to ensure smooth progress. By signing this agreement, all team members confirm their dedication to working diligently, responsibly, and collaboratively for the successful completion of the project.

Full Name	**Student ID**	**Signature**
Kanukuntla Ranadheer	A00086685	
Kethineni Neeraj Krishna	A00076167	
Venkatasubbaiah Chowdary Kunku	A00075088	
Bhargava Sai Bellamkonda	A00073767	

GitHub Link

Task Board Link